

01 MI

Universal ML Interview Notes

Shape Rules

Always ask:

- N = samples
- D = features
- C = classes

Complexity Rules

DSA: $O(N)$

ML: Often $O(ND)$ because each sample has D features.

Bias vs Variance

High Bias

- Too simple.
- Misses patterns.
- Underfitting.

High Variance

- Too flexible.
- Learns noise.
- Overfitting.

Daily Oral Drill

For every algorithm answer:

1. What problem does it solve?
2. Supervised or unsupervised?
3. Loss function?
4. Complexity?
5. Biggest weakness?
6. One improvement?

If you can answer those six questions quickly, you're usually interview-ready on that topic.

These are the notes I'd personally want in front of me 30 minutes before a Google ML coding interview.

Volume 1: Linear Regression & Gradient Descent

The Learning Problem

Machine learning starts with a simple question:

Given examples of inputs and outputs, how do we learn a rule that generalizes to unseen data?

Why not memorize points?

Memorization gives zero error on training samples but usually fails on new inputs.

A model should capture pattern, not store a lookup table.

Why fit a line?

A line is the simplest useful relationship between input and output.

Simple models are easier to train, interpret, and debug.

What is a parameter?

A parameter is a controllable value the model learns from data.

In linear regression:

- w controls how strongly x changes prediction
- b shifts prediction up/down

What is a model?

A model is a parameterized function that maps input to output.

For 1D linear regression:

$$\hat{y} = wx + b$$

Prediction Intuition

You already discovered:

$$\hat{y} = wx + b$$

Interpretation:

- w = slope knob
- b = vertical shift

Loss: How Wrong Are We?

We need a scalar score of “badness” to optimize.

For linear regression, we use Mean Squared Error (MSE):

$$\text{MSE} = (1/N) * \sum (\hat{y} - y)^2$$

Why square?

- Large mistakes are punished heavily.
- Positive and negative errors cannot cancel.

What Is a Gradient?

Not “just a derivative formula.”

Gradient answers this practical question:

If I nudge a parameter, what happens to loss?

- Positive gradient: increasing parameter increases loss.
- Negative gradient: increasing parameter decreases loss.

So the gradient is a direction signal for update.

Why Move Opposite the Gradient?

One of the most important intuitions.

- Gradient points uphill (increase in loss).
- We want downhill (decrease in loss).

So we update:

$$w_{\text{new}} = w - \text{lr} * dL/dw$$

Same for b .

Why Does x Appear in dL/dw ?

Major insight:

w affects prediction only through wx .

So if x is large, a tiny change in w causes a larger change in $\hat{y}$, which causes a larger effect on loss.

That is why x naturally appears in the gradient.

Chain Rule (Intuition First)

You do not need to treat chain rule as a trick.

Think of dependencies:

`w -> prediction -> error -> loss`

Changing `w` cannot affect loss directly.

It must flow through each intermediate node.

Chain rule = multiply local sensitivities along that path.

This dependency-tracking view is crucial later for:

- Logistic Regression
- Neural Networks
- Backpropagation

Gradient Descent Loop

1. Predict `y_hat`
2. Compute error
3. Compute gradients
4. Update parameters
5. Repeat

Learning Rate Intuition

Learning rate (`lr`) controls step size.

- Too small: very slow learning
- Medium: stable progress
- Too large: overshoot, oscillation, exploding loss

Important clarification:

- Overshooting from large `lr` is not vanishing gradient.
- Vanishing gradient is about tiny gradients, usually in deep networks.

Bias vs Variance (Introduced Here)

This is a general ML concept, not just KNN.

- High bias: model too simple, underfits patterns.
- High variance: model too flexible, overfits noise.

Linear regression can show both, depending on features, regularization, and data complexity.

Complexity

- Training: $O(ENE)$
- E = epochs
- N = samples
- D = features
- Inference: $O(D)$ per sample

Common Traps

- Forgetting learning rate
- Wrong error sign in gradient
- Mixing MSE and RMSE
- Using vector bias when scalar bias is intended

30-Second Explanation

Linear regression predicts continuous values by fitting a weighted linear relationship between input features and target values. It learns w and b by minimizing mean squared error using gradient descent, where gradients tell us how to change parameters to reduce loss.

Volume 2: Logistic Regression

Why Linear Regression Fails for Classification

Linear regression outputs any real number:

- -15
- 42
- 500

These values cannot be interpreted as probabilities.

For binary classification, we need output in $[0, 1]$.

Core Idea

1. Compute a linear score
2. Convert that score to probability

Linear score:

$$z = wx + b$$

Probability:

$$p = \text{sigmoid}(z)$$

What Is Sigmoid?

Not just a formula — think of it as a compression machine.

It maps:

- input: $(-\infty, +\infty)$
- output: $(0, 1)$

So any linear score becomes a valid probability.

What Is z ?

z is the raw model score before sigmoid.

Interpretation: **log-odds** (also called logit).

This section matters because confusion about z creates confusion in both loss and gradients.

Odds and Log-Odds Intuition

Start with probability.

Example: 75% rain chance.

- $p = 0.75$
- $1 - p = 0.25$

Odds are:

$$\text{odds} = p / (1 - p) = 0.75 / 0.25 = 3$$

Meaning: **3:1** in favor.

Why log-odds?

- Odds range: $(0, \infty)$
- Log-odds range: $(-\infty, +\infty)$

Linear models naturally produce values on $(-\infty, +\infty)$, so predicting log-odds is a perfect fit.

Then sigmoid converts log-odds back into probability.

Why Not MSE?

For classification, MSE is usually a poor fit:

- optimization can be less stable
- probability calibration behavior is weaker

Use **cross-entropy loss** instead.

Cross-Entropy Intuition

Think in four cases:

- Correct + certain -> tiny loss
- Correct + unsure -> moderate loss
- Wrong + unsure -> moderate/high loss
- Wrong + certain -> huge loss

This is exactly the behavior we want from a classification loss.

The "Magic" Gradient: $p - y$

A favorite interview result:

For sigmoid + cross-entropy, gradient w.r.t. z simplifies to:

$$p - y$$

Why this is powerful:

- elegant derivation

- efficient implementation
- easy debugging signal

This simplification comes from combining:

- sigmoid derivative
- cross-entropy derivative

Training Loop

1. Compute $z = wx + b$
2. Compute $p = \text{sigmoid}(z)$
3. Compute cross-entropy loss
4. Compute gradients ($p - y$ core term)
5. Update w , b
6. Repeat

Complexity

- Training: $O(ND)$
- Inference: $O(D)$ per sample

Common Traps

- Forgetting sigmoid in forward pass
- Taking $\log(0)$ (fix with clipping)
- Using predicted class instead of probability inside loss
- Confusing logit (z) with probability (p)

30-Second Explanation

Logistic regression performs binary classification by modeling log-odds as a linear function of features and converting that score into probability through sigmoid. It learns parameters by minimizing cross-entropy loss, with the key gradient term simplifying to $p - y$.

Volume 3: K-Means

Why Clustering Exists

Sometimes we have data with no labels.

We still want structure:

- natural groups
- segments
- prototypes

K-Means is a simple way to discover such groups automatically.

Core Intuition

Two principles repeat until convergence:

1. Each point belongs to nearest centroid.
2. Each centroid should be the average of its assigned points.

Assignment vs Update (Critical Distinction)

You struggled with this before, so separate them clearly.

Assignment Step

Each point chooses nearest centroid by distance (usually Euclidean).

Keyword: **distance**.

Update Step

Each centroid moves to mean of points assigned to that cluster.

Keyword: **mean position**, not mean distance.

Why Mean?

K-Means minimizes squared distances.

For squared-error objectives, the arithmetic mean is the minimizer.

Why not median?

- Median minimizes absolute distance (**L1**), not squared distance (**L2**).

Why not mode?

- Mode is not a geometric center for continuous vectors.

K-Means Objective

K-Means minimizes:

Within-Cluster Sum of Squares (WCSS)

$$WCSS = \sum ||x_i - \mu_{\{c_i\}}||^2$$

Important:

- It has a clear loss/objective.
- But classical K-Means is not gradient descent.

It uses alternating optimization:

- assignment step
- centroid update step

Why Outliers Break K-Means

Your `100,100` style example is perfect.

One extreme point can pull a centroid far away because mean is sensitive to outliers.

Result:

- distorted cluster centers
- worse assignments for normal points

Elbow Method

To choose `K` :

1. Run K-Means for multiple `K` values.
2. Plot `K` vs WCSS.
3. Look for an “elbow” where gain starts diminishing.

Engineering weakness you spotted:

- This requires many full runs, which can be expensive.

Complexity

$$O(INKD)$$

- `I` = iterations
- `N` = samples
- `K` = clusters
- `D` = features

Strengths

- Simple and fast baseline
- Easy to interpret centroids
- Works well for compact, spherical clusters

Weaknesses

- Must choose **K**
- Sensitive to initialization
- Sensitive to outliers
- Assumes roughly spherical/equal-variance clusters

Common Traps

- Averaging distances instead of points
- Updating centroid with all points, not assigned points
- Forgetting reassignment after centroid move
- Hardcoding cluster count without validation

30-Second Explanation

K-Means is an unsupervised clustering algorithm that alternates between assigning points to their nearest centroid and updating centroids to the mean of assigned points. It minimizes within-cluster squared distance (WCSS) through iterative assignment-update steps until convergence.

Volume 4: KNN

What Problem Does It Solve?

KNN performs supervised classification using nearby labeled examples.

Given a new point, it predicts label from the labels of nearest training points.

Why It Is Called "Lazy"

KNN has almost no explicit training phase.

It mostly stores the dataset.

Computation is deferred to prediction time.

That is why KNN is called a **lazy learner**.

Prediction Flow

1. Compute distance to all training points
2. Select **K** nearest points
3. Gather their labels
4. Majority vote
5. Return predicted class

Why Inference Is Expensive

For each query, KNN compares against all stored points.

That means prediction cost scales with training set size.

K = 1

- Very flexible decision boundary
- Can memorize noise
- Low bias, high variance

Large **K**

- Smoother decision boundary
- Uses broader/global neighborhood

- High bias, low variance

Bias-Variance Tradeoff via KNN

KNN is one of the clearest ways to explain bias/variance:

- Small K : reacts strongly to local fluctuations -> variance risk
- Large K : oversmooths and misses local structure -> bias risk

Complexity

Per prediction:

- Distance computation: $O(ND)$
- Full sort approach: $O(ND + N\log N)$
- Using partial selection (`argpartition` style): near $O(ND)$

Where:

- N = training samples
- D = features

Practical Notes

- Feature scaling is important (distance-based method)
- Class imbalance can skew voting
- Weighted voting can improve robustness

Common Traps

- Voting on neighbor indices instead of neighbor labels
- Using fixed threshold instead of nearest-neighbor voting
- Forgetting to use exactly K nearest points
- Hardcoding label assumptions

30-Second Explanation

KNN predicts a sample's class by finding the K nearest labeled training points and using majority vote. It is easy to understand and often strong as a baseline, but inference is expensive because each prediction compares against the full dataset.

Sequence Models: From Feed Forward Networks to Attention

The Big Picture

Every new architecture was invented to solve a limitation of the previous one:

```
Feed Forward Network
  ↓ No memory, fixed-size inputs
RNN
  ↓ Vanishing gradients, sequential computation
LSTM
  ↓ Still sequential, fixed memory capacity
GRU
  ↓ Simplified LSTM
Seq2Seq
  ↓ Entire sentence compressed into one context vector
Attention
  ↓ Removes context vector bottleneck
Transformer (covered separately)
```

1. Feed Forward Networks

Strengths

Excellent for image classification, regression, tabular data — any fixed-size input.

```
Image → Cat / Dog
House Features → House Price
```

Fundamental Limitation

Feed Forward Networks have **no memory**. Each prediction depends only on the current input.

- Fixed input size
- No notion of sequence
- No notion of time

```
"Dog bites man." vs "Man bites dog."
```


Both contain identical words but completely different meanings. A feed-forward network has no natural mechanism to understand ordering.

Problem Statement

How do we build a neural network that **remembers** previous words?

2. Recurrent Neural Network (RNN)

Idea

Instead of processing only the current word, also feed the previous hidden state back into the network.

```
Instead of:  $h = f(x)$   
We compute:  $h_t = f(x_t, h_{t-1})$   
  
where:  
   $x_t$     = current word  
   $h_{t-1}$  = previous memory
```

The hidden state becomes a **compressed summary** of everything seen so far.

Advantages

- Introduces memory
- Can process variable-length sequences
- Suitable for language, speech, time series

Problems

1. Information Bottleneck

Only one hidden state stores the entire history. Every new word updates the same memory. Eventually older information gets overwritten.

```
Trying to summarize a tweet and a novel using the same-sized memory.
```

2. Vanishing Gradient

During training, the gradient must travel backwards through many time steps. For long sequences, learning signals gradually disappear.

```
"The movie was not ...(100 words later)... good."  
The model often forgets the word "not".
```

3. Sequential Computation

Each hidden state depends on the previous one:

```
h1 → h2 → h3 → h4
```

Computation cannot be parallelized. GPUs remain largely underutilized.

3. Long Short-Term Memory (LSTM)

Motivation

RNN has memory, but it has no control over **what to remember** and **what to forget**. Everything is mixed together.

Main Idea

Introduce **controlled memory**. Instead of blindly updating memory, learn what to forget, what to remember, and what to expose.

Components

Component	Role
Cell State	Long-term memory. Carries important information across many time steps.
Hidden State	Working memory. Used for the current prediction.

Gates

Gate	Question	Function
Forget Gate	"What should I forget?"	Removes unnecessary information
Input Gate	"What should I remember?"	Stores important new information
Output Gate	"What should I reveal?"	Controls which information is exposed to the next layer

Advantages

- Greatly reduces vanishing gradients
- Learns much longer dependencies
- Controlled memory updates

Remaining Problems

- Still sequential
 - Still uses a fixed-size memory representation
 - Still difficult to parallelize
-

4. Gated Recurrent Unit (GRU)

Observation

LSTM works well, but is fairly complicated.

Idea

Simplify LSTM:

- Removes the separate cell state
- Uses fewer gates
- Fewer parameters
- Faster training

Interview Summary

	LSTM	GRU
Expressiveness	More expressive	Simpler
Long dependencies	Better	Often similar
Speed	Slower	Faster
Parameters	More	Fewer

5. Sequence-to-Sequence (Seq2Seq)

Goal

Translate one sequence into another.

```
Input Sentence
↓
Encoder RNN
↓
Context Vector
↓
Decoder RNN
↓
Output Sentence
```

The encoder converts the input sentence into a single **context vector**. The decoder generates the output sentence from that vector.

Problem

Entire sentence → One vector. Large information bottleneck. Long sentences lose important information.

```
Trying to compress an entire paragraph into a single vector.
```

6. Attention

Key Insight

The decoder no longer depends only on one context vector. Instead, it has access to **every encoder hidden state**.

While generating each output token, the decoder decides which encoder words deserve attention.

Example

```
Input: "The movie was not good."

Generating "good":
The model attends strongly to "not"
instead of treating every previous word equally.
```

Advantages

- Removes the single context bottleneck
- Long sentences become much easier to translate
- The decoder can dynamically retrieve information from any part of the input

Why Attention Changed Everything

Instead of: "I must remember everything."
Attention: "I'll simply look at what I need."

This idea eventually led to Transformers.

Evolution Summary

Architecture	Innovation	Problem Solved	Remaining Problem
Feed Forward	Fixed-size mapping	Basic classification	No memory
RNN	Recurrent hidden state	Memory for sequences	Vanishing gradients, sequential
LSTM	Gates (forget/input/output)	Long-term dependencies	Still sequential
GRU	Simplified LSTM	Fewer parameters	Same limitations
Seq2Seq	Encoder-decoder	Sequence translation	Context vector bottleneck
Attention	Dynamic retrieval	Removes bottleneck	Still uses recurrence
Transformer	Self-attention	Removes recurrence	(covered separately)

Interview Takeaways

- **Feed Forward:** No memory. Fixed-size inputs only.
- **RNN:** Introduces memory using recurrent hidden states. Main problems: vanishing gradients, information bottleneck, sequential execution.
- **LSTM:** Controlled memory through forget, input, and output gates. Greatly improves long-term dependency learning.
- **GRU:** Simpler, faster version of LSTM with fewer parameters.
- **Seq2Seq:** Encoder-decoder architecture for sequence generation. Limited by a single context vector.
- **Attention:** Allows the decoder to dynamically retrieve information from any encoder hidden state. Eliminates the context bottleneck and lays the foundation for Transformers.

Classical ML: Decision Trees, Random Forest, XGBoost & PCA

1. Decision Trees

Core Idea

Recursively partition the feature space by selecting splits that maximize information gain or minimize impurity.

```
Example:
Is Weather = Rain?
├─ Yes → Don't Play
└─ No
    │
    ▼
Is Humidity High?
├─ Yes → Don't Play
└─ No → Play
```

- Each **internal node** = a question (feature split)
- Each **branch** = an answer
- Each **leaf** = final prediction

Impurity

Node Type	Composition	Impurity
Pure	All same class	0
Mixed	Multiple classes	High

Objective: recursively create pure leaf nodes.

Gini Impurity

Most commonly used split criterion:

$$G = 1 - \sum p_i^2$$

Example: Yes=0.5, No=0.5

$$G = 1 - (0.25 + 0.25) = 0.5$$

Lower Gini = better split.

Entropy

Alternative impurity measure:

$$\text{Entropy} = -\sum p_i \log(p_i)$$

$$\text{Information Gain} = \text{Entropy}_{\text{before}} - \text{Entropy}_{\text{after}}$$

The tree selects the split with the highest information gain.

Continuous Features

Decision Trees automatically search for thresholds:

Age < 25? Age < 30? Age < 35?

The threshold producing the purest children is selected.

Preventing Overfitting

Control	Description
<code>max_depth</code>	Limit tree depth
<code>min_samples_split</code>	Minimum samples to split a node
<code>min_samples_leaf</code>	Minimum samples in a leaf
<code>max_leaf_nodes</code>	Maximum number of leaves

Complexity

- Training: $O(n \cdot d \cdot \log n)$
- Prediction: $O(\text{depth})$

Advantages

- Easy to interpret (visual tree)
- Handles nonlinear relationships
- Works with numerical and categorical features
- Minimal preprocessing (no scaling needed)

Disadvantages

- High variance — small data changes produce different trees
- Overfits easily without pruning

Interview Q&A

Why don't trees require feature scaling?

Thresholds compare feature values. Scaling changes `Age < 25` to `Age < 2500` — ordering remains identical, so scaling has no effect.

Gini vs Entropy?

Gini is computationally faster and the common default. Entropy has information-theoretic interpretation. Often produces similar trees.

2. Random Forest

Motivation

A single Decision Tree has high variance. Random Forest reduces variance by **averaging many trees**.

Core Idea

Train many independent trees and combine predictions:

- **Classification:** Majority Vote
- **Regression:** Average Prediction

Two Sources of Randomness

1. Bootstrap Sampling

Each tree is trained on a bootstrap sample (sampling with replacement):

```
Original:  1 2 3 4 5
Bootstrap: 2 5 5 1 3  (some repeat, some omitted)
```

2. Random Feature Selection

At each split, only a random subset of features is considered. Different trees learn different decision boundaries.

Out-of-Bag (OOB) Error

~1/3 of training samples are not selected for each tree. Those samples become an internal validation set — provides performance estimate without a separate validation split.

Advantages

- Lower variance
- Less overfitting
- High accuracy
- Robust to noisy features

Disadvantages

- Larger model (many trees)
- Slower inference
- Harder to interpret

Random Forest vs Decision Tree

	Decision Tree	Random Forest
Trees	One	Many
Variance	High	Lower
Interpretability	Easy	Less interpretable
Overfitting	Easy	Better generalization

3. XGBoost (Extreme Gradient Boosting)

Motivation

Random Forest builds trees independently. XGBoost builds trees **sequentially** — each new tree corrects the mistakes of the previous trees.

Workflow

```
Tree 1 → Compute Errors → Tree 2 learns residuals → Update Predictions  
→ Tree 3 corrects remaining errors → Continue until convergence
```

Final Prediction

```
Prediction = Tree_1 + Tree_2 + Tree_3 + ...
```

Residual Learning

$$\text{Residual} = \text{Actual} - \text{Prediction}$$

The next tree predicts the residual, not the original target. Classification uses gradients of the loss function instead of simple residuals — hence **Gradient Boosting**.

Learning Rate

Each tree contributes only a fraction:

$$\text{Prediction} = \text{Old} + \eta \times \text{NewTree}$$

Small learning rates improve generalization.

Regularization

XGBoost explicitly penalizes model complexity:

$$\text{Objective} = \text{Loss} + \text{Regularization}$$

Why "Extreme"?

Engineering improvements: regularization, efficient tree construction, missing value handling, parallel split finding, memory optimization.

Advantages

- Excellent performance on tabular data
- Built-in regularization
- Handles nonlinear relationships
- Provides feature importance

Disadvantages

- Sequential training (slower)
- More hyperparameters
- Can overfit if not tuned

Random Forest vs XGBoost

	Random Forest	XGBoost
Trees	Independent	Sequential

	Random Forest	XGBoost
Training	Parallel	Sequential
Tuning	Less	More
Noise robustness	Better	Can fit noise if overtrained
Accuracy	Strong baseline	Often highest

4. Principal Component Analysis (PCA)

Motivation

Many features are highly correlated (e.g., Height & Weight). PCA compresses correlated features into a smaller set of informative components.

Core Idea

Find new **orthogonal directions** that maximize variance — called Principal Components.

- **PC1:** Captures maximum variance
- **PC2:** Captures maximum remaining variance, orthogonal to PC1

Algorithm

```

Step 1: Center the data (subtract mean from every feature)
Step 2: Compute covariance matrix  $Cov = (1/n) X^T X$ 
Step 3: Compute eigenvectors and eigenvalues
         Eigenvectors → principal directions
         Eigenvalues  → variance explained by each direction
Step 4: Sort components by decreasing eigenvalues
Step 5: Keep top k principal components
Step 6: Project data onto these components

```

Variance Explained

Component	Variance
PC1	80%
PC2	15%
PC3	5%

Keeping PC1 + PC2 preserves 95% of total variance.

Advantages

- Dimensionality reduction
- Faster training, lower memory
- Removes multicollinearity
- Useful for visualization

Disadvantages

- Components lose interpretability
- Linear technique only
- May discard predictive low-variance features

Interview Q&A

Why center the data?

Centering removes the mean so PCA captures variance rather than absolute offsets.

Why doesn't PCA use labels?

PCA is unsupervised — operates only on the feature matrix X . Target labels are completely ignored.

Why can PCA hurt accuracy?

PCA removes low-variance components, but low-variance features may still be highly predictive. Always validate on downstream model performance.

Key Interview Takeaways

Algorithm	Key Concept	Strength	Weakness
Decision Tree	Recursive splitting, Gini/Entropy	Interpretable, nonlinear	High variance, overfits
Random Forest	Bootstrap + random features + voting	Low variance, robust	Less interpretable, slower
XGBoost	Sequential residual/gradient boosting	Highest accuracy, regularized	Sequential, more tuning
PCA	Eigenvectors of covariance matrix	Dimensionality reduction	Unsupervised, loses interpretability

Final Classical ML Topics: Cross Validation, PR Curve & Naive Bayes

1. Cross Validation

Motivation

A single train-test split may produce misleading evaluation results:

```
80% Train | 20% Test → Accuracy = 91%  
Was this split representative?
```

Cross Validation evaluates the model over **multiple train-validation splits** for a more reliable estimate of generalization.

K-Fold Cross Validation

```
K = 5  
Split: Fold1 | Fold2 | Fold3 | Fold4 | Fold5  
  
Round 1: Train [2,3,4,5]  Validate [1]  
Round 2: Train [1,3,4,5]  Validate [2]  
...  
Round 5: Train [1,2,3,4]  Validate [5]
```

Every fold serves as validation exactly once.

Final Performance

```
Validation accuracies: 90%, 92%, 91%, 89%, 93%  
Final score: Mean = 91%
```

Much more stable than a single split.

Advantages

- Better estimate of generalization
- Efficient use of limited data
- Useful for hyperparameter tuning
- Reduces dependence on one random split

Disadvantages

- Computationally expensive (K training rounds)
- Rarely used for very large deep learning models

Why Not For LLM Training?

Training a modern LLM already requires enormous compute. Repeating training K times is generally infeasible.

2. Precision-Recall Curve

Motivation

Accuracy becomes misleading for imbalanced datasets:

```
1000 Patients: 990 Healthy, 10 Cancer
Model predicts "Everyone Healthy" → Accuracy = 99%
Yet the model completely fails to detect cancer.
```

Precision

Of all predicted positives, how many were actually positive?

```
Precision = TP / (TP + FP)
```

High Precision → Few False Positives

Think: Can I trust the model when it predicts Positive?

Recall

Of all actual positives, how many did the model detect?

```
Recall = TP / (TP + FN)
```

High Recall → Few False Negatives

Think: Did the model find all the positive examples?

Memory Trick

Metric	Denominator	Thinks About
Precision	Predicted Positives (TP + FP)	"Of what I predicted as positive..."

Metric	Denominator	Thinks About
Recall	Real Positives (TP + FN)	"Of what is actually positive..."

Threshold Trade-off

Threshold	Precision	Recall	Effect
High	↑	↓	Only highly confident predictions accepted
Low	↓	↑	More positives detected, but more false positives

PR Curve

Each classification threshold produces a (Precision, Recall) pair. Plotting these produces the PR Curve. **Larger area under the curve = better classifier.**

PR vs ROC

	ROC Curve	PR Curve
Best for	Balanced datasets	Imbalanced datasets

3. Naive Bayes

Motivation

Email contains: "Free", "Money", "Lottery"
Spam or Not Spam?

Naive Bayes computes $P(\text{Spam} \mid \text{Email})$ using Bayes' Theorem.

Bayes Theorem

$$P(A|B) = P(B|A) \times P(A) / P(B)$$

For classification:

$P(\text{Class} \mid \text{Features}) \rightarrow$ predict class with highest posterior

Why "Naive"?

Assumes all input features are **conditionally independent**:

$$P(\text{Spam} \mid \text{Free, Money, Lottery}) \propto P(\text{Spam}) \times P(\text{Free}|\text{Spam}) \times P(\text{Money}|\text{Spam}) \times P(\text{Lottery}|\text{Spam})$$

This assumes "Free" is independent of "Money" — rarely true, but the classifier often performs surprisingly well.

Training

Estimate:

- **Class Prior:** $P(\text{Class})$
- **Feature Likelihood:** $P(\text{Feature} \mid \text{Class})$

Prediction

$$P(\text{Class}) \times P(\text{Feature}_1|\text{Class}) \times P(\text{Feature}_2|\text{Class}) \times \dots$$

→ Choose class with highest posterior probability

Types

Type	Features	Use Case
Gaussian	Continuous numerical	Real-valued features
Multinomial	Word counts	Text classification (most common)
Bernoulli	Binary (present/not)	Binary feature presence

Advantages

- Extremely fast
- Requires little training data
- Strong baseline for text classification
- Easy to implement

Disadvantages

- Strong independence assumption
- Usually less accurate than modern discriminative models

Why Does It Still Work?

Although calculated probabilities may be inaccurate, the **correct class often still receives the highest score**. Classification depends on ranking, not perfectly calibrated probabilities.

Precision vs Recall Cheat Sheet

Metric	Formula	Measures
Precision	$TP / (TP + FP)$	Of predicted positives, how many were correct?
Recall	$TP / (TP + FN)$	Of actual positives, how many were found?

Classical ML Summary

Topic	Core Idea
Decision Tree	Recursive feature splitting using impurity reduction
Random Forest	Ensemble of independent trees using bagging
XGBoost	Sequential boosting using residual correction
PCA	Dimensionality reduction through maximum variance
Cross Validation	Reliable evaluation using multiple train-validation splits
PR Curve	Evaluation metric for imbalanced classification
Naive Bayes	Probabilistic classifier using Bayes' Theorem